

Trabajo Práctico Final

A la caza de las vinchucas

Integrantes

- Kevin Enzo Paz (kevinenzopaz@hotmail.com)
- Maximiliano Joaquín Souto (max.xies12@gmail.com)
- Nicolás Alfredo Alderete (nicolas.alderete.2002@gmail.com)

Patrones de Diseño

State

Para representar los estados que puede tener una muestra utilizamos el patrón State.

Los roles que representa son:

- **Context:** Muestra
- **State:** IEstadoMuestra
- **ConcreteState:** MuestraNoVerificada, MuestraEnVerificacion, MuestraVerificada.

Observer

Para representar las organizaciones que pueden suscribirse a la carga y validación de muestras en una zona de cobertura utilizamos el patrón Observer.

Los roles que representa son:

- **Subject:** ZonaDeCobertura
- **Observer:** IObserver
- **ConcreteObserver:** Organización

Composite

Utilizamos el patrón Composite para representar filtros de búsqueda de muestras. Lo utilizamos para poder realizar búsquedas con filtros combinados.

Los roles que representa son:

- **Component:** IBuscadorMuestras
- **Composite:** BuscadorCompuesto
- **Leaves:** BuscadorFechaCreacionMuestra,
BuscadorFechaUltimaOpinion, BuscadorTipoInsecto, BuscadorAnd,
BuscadorOr.

Decisiones de Diseño

Elegimos el patrón State para representar la muestra para poder cambiar su compartamiento en tiempo de ejecución, el observer para las organizaciones lo elegimos para que las organizaciones puedan suscribirse a las zonas de cobertura y así ser notificadas cuando se sube o valida una muestra, por último, se eligió el patrón composite para poder realizar búsquedas compuestas.

Detalles de Implementación

Se crearon las clases GestorDeUsuario y CalculadorGeodesico para no romper los principios de SOLID al dar responsabilidades de más al sistema y las ubicaciones.

Pensamos las opiniones como Enums porque necesitábamos que sea un grupo reducido de opciones, por lo que utilizar Strings no era viable.

Si bien inicialmente se evaluó aplicar el patrón State para modelar los niveles de los usuarios (Básico, Experto, Especialista), se decidió no utilizarlo dado que el comportamiento entre los distintos niveles es muy limitado y poco dinámico. En cambio, se optó por una solución más simple: definir un método esExperto() en la clase Usuario que encapsula lo necesario para distinguir un usuario experto de uno que no lo es. Esta decisión evita una complejidad innecesaria, reduce el número de clases, y mejora la mantenibilidad del sistema, sin perder expresividad. El patrón State fue reservado para los casos donde la variación de comportamiento entre estados lo justificaba claramente, como en los estados de la Muestra.

